

Amazon MSK Automation Upgrade using Golang and Jenkins

Project progress report submitted to

MANIPAL ACADEMY OF HIGHER EDUCATION

For Partial Fulfillment of the Requirement for the Award of the

Degree

of

Bachelor of Technology

in

Computer and Communication Engineering

by

Kavya Goel

Reg. No. 200953112

Under the guidance of

Akshay K.C.

Assistant Professor- Senior Scale

Department of I & CT

Manipal Institute of Technology

Manipal, India

Manoj Kumar Melath

Section Manager

HPE Greenlake

HPE

Bangalore, India



MANIPAL INSTITUTE OF TECHNOLOGY

MANIPAL

(A constituent unit of MAHE, Manipal)

April 2024

1. Abstract

This project focuses on automating the upgrade process of Amazon Managed Streaming for Apache Kafka (MSK) versions within the HPE GreenLake environment. Through the implementation of Jenkins pipelines, the project aims to streamline version checks, testing, and deployment procedures, ensuring prompt upgrades to supported MSK versions. The project's scope encompasses integration with existing infrastructure, compatibility checks, error handling mechanisms, documentation creation, and staff training, aiming to mitigate challenges associated with manual upgrades, such as downtime risks and operational inefficiencies.

By automating the upgrade process, the project offers numerous benefits, including increased efficiency, reliability, cost savings, improved security, scalability, and a competitive advantage. These advantages stem from the elimination of manual errors, faster deployment cycles, and the ability to leverage the latest features and enhancements provided by updated MSK versions. Ultimately, the project endeavours to optimize the MSK upgrade experience within the HPE GreenLake environment, enhancing overall operational effectiveness and ensuring seamless integration with evolving business needs.

2. Introduction

In the rapidly evolving landscape of real-time data processing, Amazon Managed Streaming for Apache Kafka (Amazon MSK) has become indispensable for organizations striving to harness the potential of data streaming. However, ensuring the optimal performance and security of Kafka-based infrastructures necessitates periodic upgrades of MSK clusters to align with the latest Kafka versions. The manual execution of these upgrades poses challenges in terms of labor intensity, error susceptibility, and operational disruptions. To address these concerns, this project embarks on automating the MSK upgrade process using an integrated approach leveraging Jenkins Pipelines, Golang, and Terraform.

By harnessing Jenkins Pipelines, a widely adopted automation framework for continuous integration and delivery (CI/CD), this project endeavors to establish a streamlined and repeatable process for upgrading MSK clusters. Integrating Golang, a powerful programming language known for its efficiency and concurrency, enables the creation of custom tools and utilities to enhance the automation workflow. Additionally, Terraform, an infrastructure-as-code tool, facilitates the provisioning and management of cloud resources, ensuring consistency and reproducibility across MSK cluster configurations.

This integrated approach not only reduces the manual overhead associated with MSK upgrades but also ensures consistency, reliability, and scalability throughout the upgrade lifecycle. By adopting agile methodologies, the project aims to adapt and respond to evolving requirements, prioritize tasks based on business value, and foster collaboration among cross-functional teams. Through iterative sprints, regular reviews, and retrospectives, the project is poised to deliver tangible value by accelerating the pace of MSK upgrades, minimizing downtime, and ensuring the seamless delivery of streaming data for critical business operations.

3. Literature Survey

Mysari et al [\[1\]](#) outlined the process of building and integrating software using Jenkins with pipeline methodology and deploying it with Ansible. It provided a basic framework for automating the deployment pipeline, streamlining the development process, and ensuring consistency in software releases. By leveraging Jenkins and Ansible, organizations can achieve continuous integration and deployment, resulting in faster delivery cycles and improved software quality.

The paper [\[2\]](#) delves into the intricacies of Apache Kafka's structure and workflow, proposing a queueing-based packet flow model to predict performance metrics for Kafka cloud services. By analysing the system's architecture and packet flow dynamics, the study offers insights into optimizing Kafka deployments for improved performance and scalability in handling large-scale data streams.

H. Wu, Z. Shang, and K. Wolter [\[3\]](#) focuses on predicting the reliability of Apache Kafka under different configurations and network conditions. By assessing various factors influencing Kafka's reliability, such as message replication and partitioning strategies, organizations can make informed decisions to ensure consistent and reliable delivery of streaming data, thereby enhancing the overall robustness of their data pipelines.

The study [\[4\]](#) explores the benefits and challenges of automation in cloud computing environments, emphasizing the importance of automated workflows for efficient resource management and scalability. It discusses the opportunities presented by automation technologies in improving operational efficiency, reducing manual effort, and enhancing agility in cloud infrastructure management.

The survey [\[5\]](#) provides a comprehensive overview of Apache Kafka's features, capabilities, and use cases in stream processing for big data applications. It underscores the significance of timely version upgrades to leverage new features and optimizations, enabling organizations to harness the full potential of Kafka for real-time data processing and analytics.

H. Ali and M. Z. Abdullah [\[6\]](#) explores various automated deployment strategies, including continuous integration/continuous deployment (CI/CD) pipelines, for effectively managing and scaling enterprise applications in dynamic cloud environments. By dynamically deploying and auto-scaling applications based on workload demands, organizations can optimize resource utilization, enhance performance, and adapt to changing business needs efficiently.

This literature review [\[7\]](#) investigates best practices and challenges associated with implementing Infrastructure as Code (IaC) for automating infrastructure provisioning and configuration. It emphasizes the role of automation in maintaining consistency, reducing manual errors, and facilitating collaboration between development and operations teams in the DevOps culture.

4. Problem Definition

The project responds to the pressing demand for automating Amazon Managed Streaming for Apache Kafka (Amazon MSK) cluster upgrades. With Kafka versions continuously evolving and older iterations approaching end-of-life support, timely upgrades are crucial for compatibility, security, and feature access. Yet, manual execution of these upgrades presents multifaceted challenges, including complex coordination across clusters, ensuring client application compatibility, and the absence of standardized automation tools.

To address these challenges, the project focuses on key areas: assessing optimal partitioning per broker, configuring crucial cluster settings like replication factor and in-sync replica, and verifying client application compatibility. Specifically targeting the upgrade of MSK versions 2.5.1 and 3.3.2, the project aims to develop automated workflows using Jenkins Pipelines, Golang, and Terraform. This integrated approach streamlines the upgrade process, minimizes errors, and ensures timely upgrades, bolstering the reliability, security, and performance of Kafka-based data streaming infrastructures amidst evolving operational landscapes.

5. Objective

1. Upgrade Execution Automation: The primary objective is to automate the execution of the upgrade process, ensuring seamless orchestration and adherence to industry standards. This involves developing and implementing automated workflows using Jenkins pipelines to initiate and manage the upgrade process efficiently.

2. Post-Upgrade Validation and Testing: The focus is on validating the successful completion of the upgrade and conducting comprehensive testing to ensure the functionality and performance of the Kafka cluster post-upgrade. This includes thorough testing of application functionality and performance to prevent any adverse impacts on critical business operations.

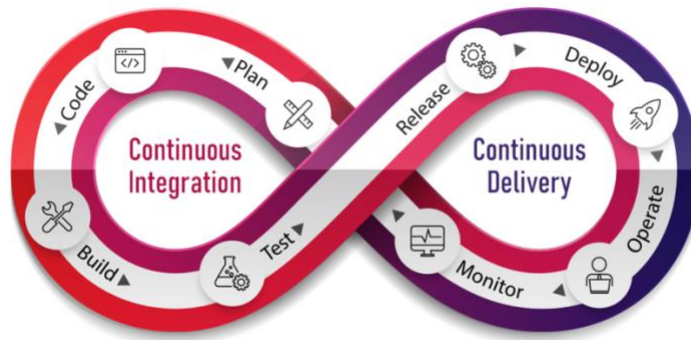
3. Continuous Monitoring and Anomaly Detection: Implementing continuous monitoring mechanisms to detect anomalies and issues post-upgrade is essential. The objective is to promptly identify and address any issues to maintain cluster stability and performance, thereby ensuring uninterrupted data streaming operations.

4. Rollback Mechanism Implementation: Despite meticulous planning, unforeseen issues may arise during the upgrade process. The objective is to implement an automated rollback mechanism to swiftly revert to the previous stable state in case of any critical issues, minimizing downtime and ensuring data integrity within the Kafka cluster.

5. Validation of Rollback Success: After executing the rollback mechanism, validating the successful restoration of cluster stability and data integrity is crucial. This ensures minimal disruption to operations and confirms the reliability of the rollback process, providing confidence in the upgrade infrastructure's resilience.

6. Methodology

DEVOPS LIFECYCLE



DevOps Lifecycle

Figure 1

We are following DevOps Lifecycle for the automation as shown in [Figure 1](#).

Following is the Agile Methodology:

1. **Sprint Planning:** During sprint planning, the team defines user stories for each objective, such as prechecks, upgrade execution, post-checks, and rollback mechanism. These user stories are prioritized based on business value and technical dependencies, and estimated story points are assigned to each. This ensures that the team has a clear roadmap for the upcoming sprint and understands the deliverables expected.
2. **Sprint Execution:** Sprint execution involves implementing user stories in iterative sprints, typically lasting 2-4 weeks. Daily stand-up meetings are conducted to discuss progress, challenges, and necessary adjustments. The team focuses on completing high-priority user stories first to ensure continuous delivery of value to stakeholders.
3. **Prechecks:** In this phase, the team's sprint goal is to ensure cluster readiness for the upgrade. Tasks include implementing checks for cluster size, partition limits, replication factor (RF) configuration, in-sync replica (ISR) configuration, and client compatibility. A sprint review is conducted to demonstrate completed precheck functionalities and gather feedback for refinement.
4. **Upgrade Execution:** The sprint goal here is to automate the initiation and execution of the upgrade process. Tasks involve developing Jenkins Pipelines for the upgrade workflow, following AWS best practices, and handling version-specific configurations. A sprint review is held to demo completed upgrade automation functionalities and gather feedback for further enhancements.
5. **Post-checks:** This phase focuses on validating upgrade completion and ensuring application functionality. Tasks include developing validation checks, functional testing, performance testing, and setting up monitoring. The sprint review showcases completed post-check functionalities and gathers feedback for iterative improvements.
6. **Rollback Mechanism:** Here, the team's sprint goal is to implement an automated rollback capability. Tasks include defining rollback procedures, developing rollback automation, and validating successful rollback. A sprint review is conducted to demonstrate the completed rollback mechanism and gather feedback for refinement.
7. **Sprint Review and Retrospective:** After completing user stories, the team conducts a sprint review with stakeholders to showcase the accomplished work. Feedback is gathered, and areas for improvement are identified. A retrospective is held to reflect on sprint execution, discuss successes, challenges, and determine action items for continuous improvement.

8. **Continuous Integration and Deployment:** The team continuously integrates developed features into the main branch and automates deployment to staging environments for testing. Integration with existing CI/CD pipelines and infrastructure is ensured to facilitate seamless delivery of new functionalities.
9. **Continuous Monitoring and Feedback:** Following deployment, the team monitors system performance, stability, and gathers user feedback. Iterative improvements are made based on feedback and performance metrics, prioritizing backlog items for future sprints to address user needs and enhance system capabilities.

[Figure 2](#) depicts the agile methodology followed.

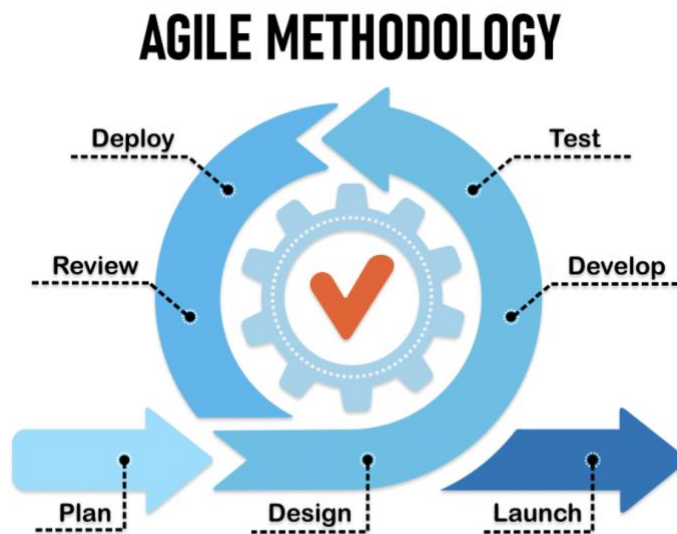


Figure 2

7. Work done So Far

1. **Prechecks Initiated:** The project has entered the critical phase of conducting prechecks across all Amazon Managed Streaming for Apache Kafka (Amazon MSK) clusters. These prechecks serve as the foundation for ensuring the clusters' readiness for the impending upgrade process. By meticulously evaluating various aspects of the clusters' configurations and environments, the team aims to identify and address any potential obstacles or compatibility issues prior to initiating the upgrade. This proactive approach not only helps mitigate risks but also contributes to a smoother transition to the new Kafka version.
2. **Jenkins Pipeline Development:** Concurrently, the development of Jenkins Pipelines is in full swing to automate the precheck procedures. These pipelines are being custom-built to streamline and standardize the validation process across all MSK clusters. By leveraging Jenkins automation capabilities, the team seeks to enhance efficiency, accuracy, and repeatability in conducting prechecks. These tailored pipelines are designed to validate critical aspects such as cluster size, partition configurations, and compatibility with the targeted Kafka version. Moreover, they will enable seamless integration with existing CI/CD workflows, ensuring a cohesive and automated upgrade process from start to finish.
3. **Cluster Size Verification:** One of the key areas of focus during prechecks is verifying the size of each MSK cluster. This entails assessing the current capacity and resource

allocation to ensure compatibility with the targeted Apache Kafka version. By accurately determining the cluster size, the team can anticipate and address any potential capacity constraints or performance bottlenecks that may arise post-upgrade. This verification step is essential for optimizing cluster performance, ensuring uninterrupted data processing, and maintaining overall system stability.

4. **Partition Analysis:** In addition to cluster size verification, the team is implementing checks to analyse the number of partitions per broker within each MSK cluster. This analysis is crucial for ensuring that the partition distribution aligns with recommended limits and best practices. By carefully examining partition configurations, the team can prevent issues such as uneven load distribution, data skew, or partition overload, which could impact the cluster's performance and scalability. Through meticulous partition analysis, the team aims to optimize cluster resource utilization and enhance overall data processing efficiency.
5. **Configuration Adjustment:** Furthermore, the team is actively adjusting various configuration settings within the MSK clusters to align them with the requirements of the new Kafka version. This includes fine-tuning parameters such as replication factor (RF), in-sync replica (ISR) settings, and multiple broker connection strings. These adjustments are essential for ensuring data integrity, fault tolerance, and high availability within the upgraded clusters. By carefully configuring cluster settings, the team can optimize performance, enhance resilience, and mitigate potential risks associated with the upgrade process.
6. **Automated Validation:** Lastly, automation efforts are focused on implementing comprehensive validation checks to verify the integrity and consistency of cluster configurations. By automating validation procedures, the team aims to minimize manual intervention and human error, while ensuring the accuracy and reliability of precheck assessments. Automated validation enables the team to quickly identify any discrepancies or anomalies in cluster configurations and promptly take corrective actions as needed. This proactive approach not only streamlines the precheck process but also lays a solid foundation for a successful and seamless upgrade of the MSK clusters.

Output:

The following is the Jenkins Pipeline created:

Branches (1)						
S	W	Name	Last Success	Last Failure	Last Duration	
		master	11 min - #1	N/A	4.6 sec	

Icon: [S](#) [M](#) [L](#)

Legend  [RSS for all](#)  [RSS for failures](#)  [RSS for just latest builds](#)

Figure 3

[Figure 3](#) represents the repository created for running Jenkins Pipeline.

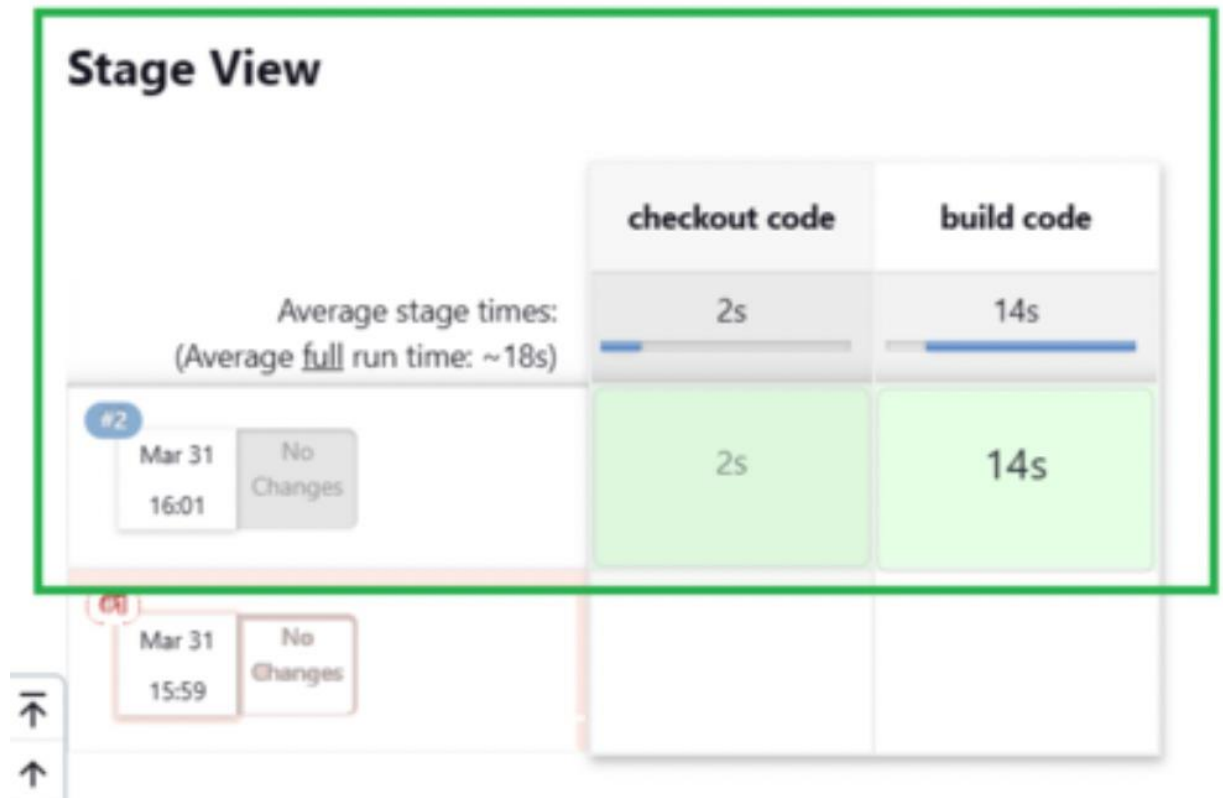


Figure 4

[Figure 4](#) depicts the stage View- visualizes the progress of various stages of our pipeline in real time. This pipeline is created to automate the partition count metrics of MSK clusters.

8. Remaining Work

1. **Client Compatibility Checks:** Efforts are underway to ensure seamless integration post-upgrade by implementing automated checks to verify the compatibility of client applications and libraries with the targeted Kafka version. Additionally, configurations on the client side are being aligned with the upgraded MSK clusters' requirements to optimize performance and reliability.
2. **MSK Upgrade Verification:** Validation procedures are being developed to confirm the successful upgrade of MSK clusters to the targeted Kafka versions, ensuring adherence to upgrade specifications. Thorough testing is being conducted to verify the functionality of all components within the clusters post-upgrade, minimizing potential issues.
3. **Post-Upgrade Validation:** Automated validation checks are being implemented to ensure the stability and integrity of the upgraded MSK clusters, reducing manual intervention and ensuring consistent performance. Functional and performance testing of Kafka-based applications and workflows is also being performed to maintain business continuity and user satisfaction.
4. **Rollback Mechanism Implementation:** Automated rollback procedures are being developed to revert to the previous Kafka version in case of critical issues during the upgrade process, ensuring rapid recovery and minimal disruption. The effectiveness of the rollback mechanism is being validated through rigorous testing to guarantee reliable fallback options.
5. **Integration and Deployment:** Developed features and functionalities are being integrated into the main branch for continuous integration, facilitating collaboration

and version control. Additionally, deployment of new capabilities to staging environments is being automated for thorough testing before production deployment, ensuring reliability and stability.

6. **Monitoring and Feedback Mechanism:** Continuous monitoring of system performance, stability, and user feedback post-deployment is being implemented to proactively identify and address any issues. Iterative improvements based on feedback and performance metrics are being made to enhance the overall reliability and usability of the upgrade automation solution.
7. **Documentation and Knowledge Sharing:** The upgrade process, including prechecks, execution steps, post-checks, and rollback procedures, is being documented for future reference. Furthermore, knowledge and best practices gained during the upgrade process are being shared across the team to foster collaboration and continuous improvement.

9. Scope

1. **Prechecks Automation:** Developing automated procedures to verify cluster readiness, such as cluster size, partition limits, replication factor (RF), in-sync replica (ISR) settings, and client compatibility checks.
2. **Upgrade Execution Automation:** Implementing Jenkins Pipelines to automate the initiation and execution of the upgrade process, following AWS best practices and ensuring smooth transition to the targeted Kafka version.
3. **Post-checks Automation:** Creating automated validation checks to confirm the successful completion of the upgrade, including verification of Kafka version, functional testing of applications, and monitoring for anomalies.
4. **Rollback Mechanism Implementation:** Developing automated rollback procedures to revert to the previous Kafka version in case of issues or failures during the upgrade process, ensuring rapid recovery and minimal downtime.
5. **Integration and Deployment:** Integrating developed features into existing CI/CD pipelines and automating deployment to staging environments for testing before production deployment.

10. Conclusion

In conclusion, the successful implementation of automated upgrade processes for Amazon Managed Streaming for Apache Kafka (MSK) within the HPE GreenLake environment marks a significant milestone in enhancing operational efficiency and reliability. By focusing on objectives such as upgrade execution automation, post-upgrade validation and testing, continuous monitoring, anomaly detection, and rollback mechanism implementation, the project has laid a robust foundation for seamless upgrade operations.

Through meticulous planning and execution, the project has mitigated risks associated with manual upgrades, minimized downtime, and ensured data integrity within the Kafka cluster. The integration of Jenkins pipelines and adherence to AWS best practices have optimized the upgrade process, enhanced reliability and efficiency while aligning with industry standards.

Moving forward, the project sets the stage for continued optimization and innovation

in managing Kafka-based data streaming infrastructures. By leveraging automated processes and continuous monitoring mechanisms, the project ensures that the Kafka clusters within the HPE GreenLake environment remain resilient, scalable, and capable of meeting evolving business needs. Overall, the project's successful outcomes underscore its contribution to enhancing operational excellence and facilitating seamless data streaming operations within the organization.

References

1. S. Mysari and V. Bejgam, "Continuous Integration and Continuous Deployment Pipeline Automation Using Jenkins Ansible," 2017.
2. H. Wu, Z. Shang, and K. Wolter, "Performance Prediction for the Apache Kafka Messaging System," 2019.
3. H. Wu, Z. Shang, and K. Wolter, "Learning to Reliably Deliver Streaming Data with Apache Kafka," 2020.
4. S. N. Srirama, I. Tverezovskyi, and J. Viil, "Dynamic Deployment and Auto-scaling Enterprise Applications on the Heterogeneous Cloud," in 2016 IEEE 9th International Conference on Cloud Computing (CLOUD), San Francisco, CA, USA, 2016, pp. 927-932.
5. H. C. Lim, S. Babu, J. S. Chase, and S. S. Parekh, "Automated control in cloud computing: challenges and opportunities," June 2019.
6. H. Ali and M. Z. Abdullah, "Recent Trends in Distributed Online Stream Processing Platform for Big Data: Survey," in 2018 1st Annual International Conference on Information and Sciences.
7. M. Basher, "DevOps: An explorative case study on the challenges and opportunities in implementing Infrastructure as code," Umeå University, Faculty of Social Sciences, Department of Informatics, 2019.

Amazon MSK Upgrade Automation

ORIGINALITY REPORT

7%

SIMILARITY INDEX

6%

INTERNET SOURCES

4%

PUBLICATIONS

0%

STUDENT PAPERS

PRIMARY SOURCES

1

impressions.manipal.edu

Internet Source

1%

2

www.semanticscholar.org

Internet Source

1%

3

www.coursehero.com

Internet Source

1%

4

web.archive.org

Internet Source

1%

5

www.essays.se

Internet Source

1%

6

academic.hep.com.cn

Internet Source

<1%

7

etd.repository.ugm.ac.id

Internet Source

<1%

8

tel.archives-ouvertes.fr

Internet Source

<1%

9

www.themuse.com

Internet Source

<1%

10 Sanjana N, Sinchana Raj, Vishalakshi Prabhu H, Sandhya S. "Real-time Event Streaming for Financial Enterprise System with Kafka", 2023 3rd Asian Conference on Innovation in Technology (ASIANCON), 2023 <1 %

Publication

11 Theofanis P. Raptis, Andrea Passarella. "A Survey on Networked Data Streaming With Apache Kafka", IEEE Access, 2023 <1 %

Publication

12 Theofanis P. Raptis, Andrea Passarella. "On Efficiently Partitioning a Topic in Apache Kafka", 2022 International Conference on Computer, Information and Telecommunication Systems (CITS), 2022 <1 %

Publication

Exclude quotes On

Exclude matches < 3 words

Exclude bibliography On